

Методические указания
по лабораторной работе №4
на тему: «Реализация сервиса для взаимодействия с разработанной базой данных»

по дисциплине: Информационная безопасность баз данных

Санкт-Петербург
2024

Цель работы

Получение навыков использования серверных языков программирования, фреймворков по работе с БД, ORM-систем

Задание

1. На основе БД, созданной в предыдущих лабораторных работах, необходимо создать сервис, который взаимодействует с разработанной БД. В качестве сервиса среди прочего могут быть разработаны:

- веб-интерфейс;
 - desktop приложение;
 - API;
 - сервисы предобработки данных, которые заполняют базу данных (обработка websocket соединений, парсинг страниц, json и др)
 - сервисы для защиты данных при их обработке, записи, считывании из/в БД (шифрование, обезличивание, сетевая защита и др.)
- (список не исчерпывает все возможные сервисы, см. п.4 в примечании)*

Дайте краткое обоснование выбранной среде разработки, языку и фреймворку, который вы используете для взаимодействия с базой данных.

2. В рамках ЛР опишите каким образом вы взаимодействуете с базой данных. С помощью каких фреймворков или языков. Продемонстрируйте несколько примеров по выборке, вставке, удалении данных из вашей БД с помощью выбранного вами фреймворка или языка программирования. Проконтролируйте корректность работы систем логирования и разграничения доступа, которые реализованы в 1-3 ЛР.

3. Для реализованного сервиса укажите кратко его основные функции, назначение, кратко опишите структуру. Дополнительно опишите и оцените методы защиты данных, реализованные вами и имеющиеся в наличии в разработанном сервисе. Выделите аналоги, недостатки и преимущества решений, которые вы реализовали.

Примечание

1. При составлении сложных запросов подбирайте отношения, подходящие для роли пользователя.
2. В отчете необходимо приводить полный листинг кода пользовательского интерфейса, достаточного для запуска исходников. Если для сборки/запуска необходимо наличие определенной среды/дополнительных инструментов, кратко их опишите.
3. В отчете необходимо кратко описать назначение и алгоритм всех разработанных модулей пользовательского интерфейса.
4. Для четвертой лабораторной работы могут быть рассмотрены альтернативы, например, составление не веб-интерфейса (как это показано в примере отчета), а составление API с набором методов, которые обращаются к БД или вместо интерфейса можно реализовать слой, который занимается предобработкой данных,

перед тем как эта информация поступает в БД. В качестве альтернативы данной лабораторной работы может быть рассмотрены другие программные реализации для окружения разработанной БД удовлетворяющие следующим требованиям:

- а) Программная реализация каким-то образом взаимодействует с разработанной БД и ее системой защиты (потребляет данные или наоборот обновляет/генерирует данные для БД);
- б) Для разработанной программной реализации также необходимо рассмотреть вопросы информационной безопасности. Например, если вы рассматриваете создание API над БД, то можно включить в отчет информацию об используемых вами алгоритмах аутентификации (например, из oauth2 и пр.)

Пример отчета по лабораторной работе №4.

В качестве сервиса, взаимодействующего с БД, решено разработать многопользовательский интерфейс для взаимодействия с базой данных

Обоснование выбора среды программирования и фреймворка. В качестве языка программирования для реализации интерфейса был выбран Python, среда программирования PyCharm. Устанавливаем среду программирования (например, с официального сайта <https://www.jetbrains.com/ru-ru/pycharm/download/?section=windows>). Установим дополнительно следующие пакеты и фреймворки с помощью будет выполняться взаимодействие с базой данных, нами были выбраны:

```
pip install sqlalchemy //ORM для python
pip install psycopg2 //драйвер для доступа в postgres DB
pip install Flask // фреймворк, который позволит реализовать простой веб-интерфейс.
```

Установим все необходимые зависимости в программное окружение. Продemonстрируем работу ORM *sqlalchemy*. Для начала работы с классами ORM нужно выполнить маппинг таблиц БД в структуру классов. Маппинг и подключение к БД представлены ниже в теле модуля «orm_test.py»:

```
from sqlalchemy import Column, Integer, Text, ForeignKey
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base
from sqlalchemy.orm import sessionmaker

Base = declarative_base()

# маппинг структуры таблиц БД в структуру классов ORM
class Teachers(Base):
    __tablename__ = 'teachers'

    teacher_id = Column(Integer(), primary_key=True)
    teacher_full_name = Column(Text)
    department = Column(Text)
    email = Column(Text)
```

```
phone = Column(Text)
consultation_time = Column(Text)
working_bulding = Column(Text)

class Subject(Base):
    __tablename__ = 'subject'

    subject_id = Column(Integer, primary_key=True)
    subject_name = Column(Text)
    subject_type = Column(Text)
    teacher_id = Column(Integer,
ForeignKey('teachers.teacher_id'))
    group_id = Column(Integer, ForeignKey('groups.group_id'))

class Speciality(Base):
    __tablename__ = "specialty"

    specialty_code = Column(Integer, primary_key=True)
    specialty_name = Column(Text)
    department = Column(Text)

class SecretData(Base):
    __tablename__ = 'secret_data'

    ID = Column(Integer, primary_key=True)
    username = Column(Text)
    secret_token = Column(Text)

class MainLog(Base):
    __tablename__ = 'main_log'

    log_item_id = Column(Integer, primary_key=True)
    operation_type = Column(Text)
    operation_date = Column(Text)
    user_operator = Column(Text)
    changed_data = Column(Text)

class Groups(Base):
    __tablename__ = 'groups'

    group_id = Column(Integer, primary_key=True)
    course_number = Column(Integer)
    admin_full_name = Column(Text)
    admin_contacts = Column(Text)
    specialty_code = Column(Integer,
ForeignKey('specialty.specialty_code'))
```

```
class Classes(Base):
    __tablename__ = 'classes'

    classes_id = Column(Integer, primary_key=True)
    classes_start_time = Column(Text)
    classes_end_time = Column(Text)
    classroom = Column(Text)
    building = Column(Text)
    subject_id = Column(Integer,
ForeignKey('subject.subject_id'))

class CompactSchedule(Base):
    __tablename__ = 'compact_shedule'

    id_schel = Column(Integer, primary_key=True)
    subject_name = Column(Text)
    classes_start_time = Column(Text)
    classes_end_time = Column(Text)
    building = Column(Text)
    teacher_full_name = Column(Text)

class Consultations(Base):
    __tablename__ = 'consultations'

    id_consult = Column(Integer, primary_key=True)
    subject_name = Column(Text)
    teacher_full_name = Column(Text)
    department = Column(Text)
    email = Column(Text)
    phone = Column(Text)
    consultation_time = Column(Text)
    working_bulding = Column(Text)

class SpecialityContacts(Base):
    __tablename__ = 'speciality_and_contacts'

    id_contact = Column(Integer, primary_key=True)
    specialty_name = Column(Text)
    group_id = Column(Integer)
    course_number = Column(Integer)
    admin_full_name = Column(Text)
    admin_contacts = Column(Text)

class SpecialityTeachers(Base):
    __tablename__ = 'speciality_and_teachers'

    id_sat = Column(Integer, primary_key=True)
    specialty_name = Column(Text)
    department = Column(Text)
```

```

group_id = Column(Integer)
subject_name = Column(Text)
teacher_full_name = Column(Text)

# непосредственно подключение к базе данных
# psw пароль индивидуальной учетки ("admin" для учетки
petya_admin,
# "roman" для учетки Romanov_student, "kozlov" для учетки
kozlov_teacher)
psw = "admin"
db = "dbs24"

# вместо petya_admin подставляем любую индивидуальную роль из БД
engine = create_engine('postgresql+psycopg2://petya_admin:' +
psw + '@localhost/' + db)
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)

db = SessionLocal()

```

Выполним несколько запросов к БД от хотя бы двух различных пользователей. При удачном запросе, выводим содержимое на экран, при отказах доступа показываем ошибки.

Запрос 1 от пользователя «Romanov_student» к представлению SpecialityTeachers .

Скрипт для выполнения запроса:

```

query_example = db.query(SpecialityTeachers).all()
for row in query_example:
    print(row.id_sat, row.specialty_name, row.group_id,
row.subject_name)

```

Результат:

```

psycopg2.errors.InsufficientPrivilege: ОШИБКА: нет доступа к представлению speciality_and_teachers

```

Запрос 2 от пользователя «Romanov_student» к таблице Teachers.

```

query_example = db.query(Teachers).all()
for row in query_example:
    print(row.teacher_id, row.teacher_full_name,
row.consultation_time, row.email)

```

Результат:

```

psycopg2.errors.InsufficientPrivilege: ОШИБКА: нет доступа к таблице teachers

```

Запрос 3 от пользователя «Romanov_student» к представлению.

```
query_example = db.query(CompactSchedule).all()
for row in query_example:
    print(row.id_schel, row.teacher_full_name, row.subject_name,
          row.classes_end_time, row.classes_start_time, row.building)
```

Результат:

```
1 Vasykov Vladimir Vasilievich Application Security 13.30 10.00 Birzhevaya, 6
2 Vasykov Vladimir Vasilievich Coding Theory 12.00 10.00 Kronverskii, 16
3 Vasykov Vladimir Vasilievich Application Security 12.00 11.00 Pesochnaya, 14
4 Dementiev Sergey Danilovich Database Security 13.10 11.40 Kronverskii, 16
5 Dementiev Sergey Danilovich Database Security 16.30 15.00 Birzhevaya, 6
6 Dementiev Sergey Danilovich Cryptography 19.30 18.00 Kronverskii, 16
7 Vasykov Vladimir Vasilievich Coding Theory 9.00 8.00 Kronverskii, 16
8 Vasykov Vladimir Vasilievich Network Security 11.40 8.20 Pesochnaya, 14
9 Kozlov Genadii Andreevich Database Security 10.00 8.20 Pesochnaya, 14
```

Запрос 4 от пользователя «petya_admin» к таблице Subject

```
query_example = db.query(Subject).all()
for row in query_example:
    print(row.subject_id, row.subject_name, row.subject_type,
          row.group_id, row.teacher_id)
```

Результат:

```
1 Database Security lection 1131 192423
2 Database Security labs 1131 827241
3 Cryptography lection 2311 827241
4 Network Security labs 2311 3277122
5 Application Security lection 3351 3277122
6 Coding Theory lection 3351 3277122
```

Покажем также примеры выполнения запросов на вставку/удаление данных с помощью ORM. Поскольку у созданных групповых ролей нет прав на редактирование информации БД, выполним эти запросы со стороны учетной записи postgres.

Запрос 5. Вставим информацию о новом преподавателе.

```
newTeacher = Teachers(
    teacher_id=11221,
    teacher_full_name='Medvedev Vsevolod',
    department='HFS',
    email='dhls@mail.ru',
    phone='9442312321',
    consultation_time='11-12, every day',
    working_bulding='Pesochnaya, 14, class 3423'
)
db.add(newTeacher)
db.commit()
```

Результат:

	teacher_id [PK] integer	teacher_full_name text	department text	email text	phone text	consultation_time text	working_building text
1	11221	Medvedev Vsevolod	HFS	dhis@mail.ru	9442312321	11-12, every day	Pesochnaya, 14, class 3423
2	192423	Kozlov Genadii Andreevich	PBKS	kga@gmail.com	(901)3235511	12-15, every monday	Kronverskil, 14, classroom 404
3	827241	Dementiev Sergey Danilovich	KTU	dsd@gmail.com	(911)8463244	18-20, every monday	Birzhevaya, 16, classroom 412
4	3277122	Vasykov Vladimir Vasilievich	TSG	vvv_new_mail@mail.ru	(922)34783823	8-14, friday	Pesochnaya, 14, class 300

Проверяем работу системы мониторинга для запроса 4.

log_item_id [PK] integer	operation_type text	operation_date text	user_operator text	changed_data text
1	1	2024-09-02 06:22:11.90381+03	postgres	['classes_id':10,'classes_start_time':'8.00','classes_end_time':'9.00','classroom':'404','building':'Kronverskil, 16','subject_id':8]
2	2	2024-09-02 06:30:12.62578+03	postgres	['teacher_id':3277122,'teacher_full_name':'Vasykov Vladimir Vasilievich','department':'TSG','email':'vvv_new_mail@mail.ru','phone':'(922)34783823','consultation_time':'8-14, friday','working_building':'Pesochnaya, 14','subject_id':11]
3	3	2024-09-02 06:34:10.67939+03	postgres	['classes_id':2,'classes_start_time':'11.00','classes_end_time':'13.30','classroom':'304','building':'Pesochnaya, 14','subject_id':11]
4	4	2024-09-03 02:51:31.39320+03	postgres	['teacher_id':11221,'teacher_full_name':'Medvedev Vsevolod','department':'HFS','email':'dhis@mail.ru','phone':'9442312321','consultation_time':'11-12, every day','working_building':'Pesochnaya, 14, class 3423']

Запрос 6. Изменим один из атрибутов в таблице Speciality

```
target_spec = db.query(Speciality).filter_by(specialty_name='IoT Security').first()
target_spec.specialty_name = 'Internet of Vehicles Security'
db.commit()
```

Результат

	specialty_code [PK] integer	specialty_name text	department text
1	103114	Information Security	KTU
2	106288	Internet of Vehicles Security	TSG

Проверяем работу лога:

3	3	D	2024-09-02 06:34:10.67939+03	postgres	['classes_id':2,'classes_start_time':'11.00','classes_end_time':'13.30','classroom':'304','building':'Pesochnaya, 14','subject_id':11]
4	4	I	2024-09-03 02:51:31.39320+03	postgres	['teacher_id':11221,'teacher_full_name':'Medvedev Vsevolod','department':'HFS','email':'dhis@mail.ru','phone':'9442312321','consultation_time':'11-12, every day']
5	5	U	2024-09-03 03:06:24.54388+03	postgres	['specialty_code':106288,'specialty_name':'Internet of Vehicles Security','department':'TSG']

В качестве основы разрабатываемого сервиса будет использована упрощенная схема взаимодействия с информацией из БД. Для созданных ранее таблиц в СУБД сформированы статические html страницы с помощью следующего python скрипта:

```
import pandas as pd

# функция формирует html код на основании выборки, отправляемой на
# вход данной функции
def sql_to_html_func(query_result, html_page_name):
    df = pd.DataFrame()

    table_header = list(vars(query_result[0]).keys())
    table_header.pop(0)
    header_frame = pd.DataFrame(table_header).T
    df = pd.concat([df, header_frame])
```

Commented [1]: Поскольку пункты ниже были добавлены поздно. Их выполнение НЕобязательно при защите 4 ЛР. Однако, их необходимо выполнить в рамках курсовой работы на оценку удовлетворительно.

```

for row_class in query_result:
    row_table = list(vars(row_class).values())
    row_table.pop(0)  # PK

    df2 = pd.DataFrame(row_table).T
    df = pd.concat([df, df2])

df.to_html('templates/'+html_page_name+'.html')

```

К этим страницам будет осуществляться многопользовательский доступ. Реализуем доступ к сгенерированным страницам с помощью flask_login, добавим метод для регистрации новых пользователей. Опишем ограничения для доступа к каждой из таблиц. Исходный код для решения вышеописанных задач ниже:

```

from flask import Flask, render_template, request, url_for, redirect
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, logout_user
import enum
from sqlalchemy import Enum

psw = "qwerty"
db_name = "dbs24"

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = 'postgresql+psycopg2://postgres:' + psw + '@localhost/' + db_name
app.config["SECRET_KEY"] = "abc"
app.debug = True
db = SQLAlchemy()

login_manager = LoginManager()
login_manager.init_app(app)

class MyRoles(enum.Enum):
    student = "student"
    teacher = "teacher"
    admin = "admin"

class Users(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)

```

```

    username = db.Column(db.String(250), unique=True,
nullable=False)
    password = db.Column(db.String(250), nullable=False)
    role = db.Column(Enum(MyRoles))

db.init_app(app)

roles_distr = {
    "student": ['compact_scheldule', 'consultations'],
    "teacher": ['speciality_and_teachers',
'speciality_and_contacts'],
    "admin": [
        'classes', 'groups', 'main_log', 'secret_data',
'speciality', 'subjects', 'teachers',
        'compact_scheldule', 'consultations',
'speciality_and_teachers', 'speciality_and_contacts'
    ]
}

with app.app_context():
    db.create_all()

@login_manager.user_loader
def loader_user(user_id):
    return Users.query.get(user_id)

@app.route('/register', methods=["GET", "POST"])
def register():
    if request.method == "POST":
        user = Users(
            username=request.form.get("username"),
            password=request.form.get("password"),
            role=request.form.get("role")
        )
        db.session.add(user)
        db.session.commit()
        return redirect(url_for("login"))
    return render_template("sign_up.html")

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":

```

```
        user = Users.query.filter_by(
            username=request.form.get("username")).first()
        if user.password == request.form.get("password"):
            login_user(user)
            return redirect(url_for("home"))
    return render_template("login.html")

@app.route("/logout")
def logout():
    logout_user()
    return redirect(url_for("home"))

@app.route("/")
def home():
    return render_template("home.html", roles_distr=roles_distr)

@app.route("/subjects")
def subjects():
    return render_template("subjects.html")

@app.route("/teachers")
def teachers():
    return render_template("teachers.html")

@app.route("/classes")
def classes():
    return render_template("classes.html")

@app.route("/groups")
def groups():
    return render_template("groups.html")

@app.route("/main_log")
def main_log():
    return render_template("main_log.html")

@app.route("/secret_data")
def secret_data():
```

```

        return render_template("secret_data.html")

@app.route("/speciality")
def speciality():
    return render_template("speciality.html")

@app.route("/compact_scheldule")
def compact_scheldule():
    return render_template("compact_scheldule.html")

@app.route("/consultations")
def consultations():
    return render_template("consultations.html")

@app.route("/speciality_and_teachers")
def speciality_and_teachers():
    return render_template("speciality_and_teachers.html")

@app.route("/speciality_and_contacts")
def speciality_and_contacts():
    return render_template("speciality_and_contacts.html")

if __name__ == "__main__":
    app.run()

```

Запустим созданный сервис

```

(venv) PS C:\Users\sergt\PycharmProjects\dbs24> python app_example.py
* Serving Flask app 'app_example'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 651-125-399

```

Зарегистрируем трех новых пользователей для каждой из ролей:

- [Login](#)
- [Create account](#)

Create an account

Username: Password: Role:

- [Login](#)
- [Create account](#)

Create an account

Username: Password: Role:

- [Login](#)
- [Create account](#)

Create an account

Username: Password: Role:

Итоговая таблица с новыми пользователями из СУБД:

	id [PK] integer	username character varying (250)	password character varying (250)	role myroles
1	1	petrov	pwd1	student
2	2	ivanova	pwd2	teacher
3	3	tkacheva	pwd3	admin

Интерфейс студента

← → ↺ ⓘ 127.0.0.1:5000

- [Login](#)
- [Create account](#)

Hi petrov.You are logged in

Please select one of the tables available to you

1. compact_schedule
2. consultations

[Logout](#)

Доступные таблицы

← → ↻ ⓘ 127.0.0.1:5000/compact_schedule

	0	1	2	3	4	5
0	building	subject_name	classes_start_time	id_schel	classes_end_time	teacher_full_name
0	Birzhevaya, 6	Application Security	10.00	1	13.30	Vasykov Vladimir Vasilievich
0	Kronverskii, 16	Coding Theory	10.00	2	12.00	Vasykov Vladimir Vasilievich
0	Pesochnaya, 14	Application Security	11.00	3	12.00	Vasykov Vladimir Vasilievich
0	Kronverskii, 16	Database Security	11.40	4	13.10	Dementiev Sergey Danilovich
0	Birzhevaya, 6	Database Security	15.00	5	16.30	Dementiev Sergey Danilovich
0	Kronverskii, 16	Cryptography	18.00	6	19.30	Dementiev Sergey Danilovich
0	Kronverskii, 16	Coding Theory	8.00	7	9.00	Vasykov Vladimir Vasilievich
0	Pesochnaya, 14	Network Security	8.20	8	11.40	Vasykov Vladimir Vasilievich
0	Pesochnaya, 14	Database Security	8.20	9	10.00	Kozlov Genadii Andreevich

← → ↻ ⓘ 127.0.0.1:5000/consultations

0	1	2	3	4	5	6		
0	subject_name	teacher_full_name	email	consultation_time	department	id_consult	phone	working_building
0	Database Security	Dementiev Sergey Danilovich	dsd@gmail.com	18-20, every monday	KTU	1	(911)8463244	Birzheva, 16, classroom 412
0	Cryptography	Dementiev Sergey Danilovich	dsd@gmail.com	18-20, every monday	KTU	2	(911)8463244	Birzheva, 16, classroom 412
0	Database Security	Kozlov Genadii Andreevich	kga@gmail.com	12-15, every monday	PBKS	3	(901)3235511	Kronverskii, 14, classroom 400
0	Network Security	Vasykov Vladimir Vasilievich	vvv_new_mail@mail.ru	8-14, friday	TSG	4	(922)34783823	Pesochnaya, 14, class 300
0	Application Security	Vasykov Vladimir Vasilievich	vvv_new_mail@mail.ru	8-14, friday	TSG	5	(922)34783823	Pesochnaya, 14, class 300
0	Coding Theory	Vasykov Vladimir Vasilievich	vvv_new_mail@mail.ru	8-14, friday	TSG	6	(922)34783823	Pesochnaya, 14, class 300

Интерфейс преподавателя:

←

→

↺

🔒

127.0.0.1:5000

- [Login](#)
- [Create account](#)

Hi ivanova. You are logged in

Please select one of the tables available to you

1. [speciality_and_teachers](#)
2. [speciality_and_contacts](#)

[Logout](#)

←

→

↺

🔒

127.0.0.1:5000/speciality_and_teachers

	0	1	2	3	4	5
0	department	subject_name	specialty_name	group_id	id_sat	teacher_full_name
0	KTU	Application Security	Information Security	3351	1	Vasykov Vladimir Vasilievich
0	KTU	Coding Theory	Information Security	3351	2	Vasykov Vladimir Vasilievich
0	KTU	Cryptography	Information Security	2311	3	Dementiev Sergey Danilovich
0	TSG	Database Security	Internet of Vehicles Security	1131	4	Kozlov Genadii Andreevich
0	TSG	Database Security	Internet of Vehicles Security	1131	5	Dementiev Sergey Danilovich
0	KTU	Network Security	Information Security	2311	6	Vasykov Vladimir Vasilievich

←

→

↺

🔒

127.0.0.1:5000/speciality_and_contacts

	0	1	2	3	4	5
0	specialty_name	group_id	admin_full_name	course_number	id_contact	admin_contacts
0	Information Security	3351	Vasiliev Andrei	3	1	phone 921-323-23-44
0	Information Security	2311	Krivov Dmitrii	2	2	mail kdmitr@gmail.com
0	Internet of Vehicles Security	1131	Ivanov Ivan	1	3	ivan@mail.ru

Интерфейс администратора:

←

→

↺

ⓘ

127.0.0.1:5000

- [Login](#)
- [Create account](#)

Hi tkacheva. You are logged in

Please select one of the tables available to you

- 1. [classes](#)
- 2. [groups](#)
- 3. [main_log](#)
- 4. [secret_data](#)
- 5. [speciality](#)
- 6. [subjects](#)
- 7. [teachers](#)
- 8. [compact_scheldule](#)
- 9. [consultations](#)
- 10. [speciality_and_teachers](#)
- 11. [speciality_and_contacts](#)

[Logout](#)

Выведем выборочно несколько таблиц, доступных администратору:

←

→

↺

ⓘ

127.0.0.1:5000/groups

	0	1	2	3	4
0	admin_contacts	group_id	admin_full_name	specialty_code	course_number
0	phone 921-323-23-44	3351	Vasiliev Andrei	103114	3
0	mail kdmitr@gmail.com	2311	Krivov Dmitrii	103114	2
0	ivan@mail.ru	1131	Ivanov Ivan	106288	1

	0	1	2	3	4
0	operation_type	operation_date	changed_data		user_operation_log_item_id
0	I	2024-09-02 06:22:11.909051+03	["classes_id":10,"classes_start_time":"8:00","classes_end_time":"9:00","classroom":"404","building":"Kronverskii,16","subject_id":6]	postgres	1
0	U	2024-09-02 06:30:12.827738+03	["teacher_id":3277122,"teacher_full_name":"Vasykov Vladimir Vasilievich","department":"TSG","email":"vvv_new_mail@mail.ru","phone":"942312321","consultation_time":"8-14, friday","working_building":"Pesochnaya, 14, class 300"]	postgres	2
0	D	2024-09-02 06:34:10.67319+03	["classes_id":2,"classes_start_time":"11:00","classes_end_time":"11:30","classroom":"304","building":"Pesochnaya,14","subject_id":1]	postgres	3
0	I	2024-09-03 02:31:31.382362+03	["teacher_id":11221,"teacher_full_name":"Medvedev Vsevolod","department":"HFS","email":"dhs@mail.ru","phone":"9442312321","consultation_time":"11-12, every day","working_building":"Pesochnaya, 14, class 3423"]	postgres	4
0	U	2024-09-03 03:06:24.543881+03	["specialty_code":10638,"specialty_name":"Internet of Vehicles Security","department":"TSG"]	postgres	5

←

→

↺

ⓘ

127.0.0.1:5000/teachers

	0	1	2	3	4	5	6
0	email	phone	working_bulding	department	teacher_id	teacher_full_name	consultation_time
0	kg@gmail.com	(901)3235511	Kronverskii, 14, classroom 404	PBKS	192423	Kozlov Genadii Andreevich	12-15, every monday
0	dsd@gmail.com	(911)8463244	Birzhevaya, 16, classroom 412	KTU	827241	Dementiev Sergey Danilovich	18-20, every monday
0	vvv_new_mail@mail.ru	(922)34783823	Pesochnaya, 14, class 300	TSG	3277122	Vasykov Vladimir Vasilievich	8-14, friday
0	dhs@mail.ru	9442312321	Pesochnaya, 14, class 3423	HFS	11221	Medvedev Vsevolod	11-12, every day

Резервную копию БД, а также полные исходники разработанного сервиса и указанных в ЛР скриптов можно найти в приложении

<https://drive.google.com/drive/folders/1t8juZJKcoRopHDSzBIH50O2-Km5W5BH?usp=sharing>

Выделим недостатки разработанного приложения с точки зрения информационной безопасности:

- разработанный алгоритм доступа не использует систему разграничения доступа и определенные в СУБД роли, а только дублирует ее в коде сервиса. В связи с этим при изменении системы разграничения доступа в СУБД, данные изменения необходимо учитывать в самом сервисе причем с изменением исходного кода.
- сервис не поддерживает выгрузку обновленных данных, добавление, изменение и удаление данных в/из субд. Для выгрузки обновленных данных необходим повторный или регулярный запуск скрипта `sql_to_html.py` из полных исходников
- все данные пользователей в том числе пароли хранятся полностью в открытом виде без шифрования и хэширования